

# Formal ILP Formulation for RunSoC 2.0

May 15, 2026

## 1 Problem Definition

This document formalizes the integer linear programming (ILP) model used for assigning and scheduling a set of tasks and their associated execution instances (jobs) on a heterogeneous multi-core platform. The model assumes a *partitioned scheduling* approach, where all jobs belonging to a specific task must execute on the same core. The model minimizes a weighted objective consisting of memory-budget overflow penalties and communication penalties between dependent tasks, subject to strict precedence and timing constraints.

## 2 Sets

$T$  := set of tasks,

$J$  := set of jobs (task instances),

$C$  := set of cores,

$K$  := set of clusters,

$D_{\text{job}} \subseteq J \times J$  := set of job execution precedence dependencies,

$D_{\text{task}} \subseteq T \times T$  := set of task dependencies (for evaluating communication penalties),

$E_t \subseteq C$  := set of eligible cores for task  $t \in T$ ,

$C_k := \{c \in C \mid c \text{ belongs to cluster } k\}, \quad \forall k \in K.$

Let  $\tau(j) \in T$  denote the parent task to which job  $j \in J$  belongs. A dependency  $(j, j') \in D_{\text{job}}$  means that job  $j$  must finish before job  $j'$  may start. A dependency  $(t_1, t_2) \in D_{\text{task}}$  indicates data flow between task  $t_1$  and task  $t_2$ , incurring a penalty if assigned to different cores or clusters.

## 3 Parameters

For each job  $j \in J$ :

$r_j$  := release time of job  $j$ ,

$d_j$  := absolute deadline of job  $j$  (if specified),

$p_j$  := base processing duration of job  $j$ .

For each task  $t \in T$ :

$m_t$  := memory demand of task  $t$ .

For each core  $c \in C$ :

$\alpha_c$  := WCET scaling factor of core  $c$ ,

$B_c^{\text{core}}$  := memory budget of core  $c$ .

For each cluster  $k \in K$ :

$$B_k^{\text{cluster}} := \text{memory budget of cluster } k.$$

Let  $\kappa(c) \in K$  denote the cluster containing core  $c$ .

The processing time of job  $j$  on core  $c$  is:

$$p_{jc} := p_j \alpha_c. \quad (1)$$

The model utilizes a sufficiently large constant  $M$  for linearizing non-overlap constraints:

$$M = \max_{j \in J} d_j \quad (\text{or maximal achievable horizon}). \quad (2)$$

Penalty weights are denoted by:

$$\begin{aligned} \lambda_{\text{core}} &:= \text{penalty weight for core-memory overflow,} \\ \lambda_{\text{cluster}} &:= \text{penalty weight for cluster-memory overflow,} \\ w_{\text{intra}} &:= \text{penalty for communication on the same core,} \\ w_{\text{intercore}} &:= \text{penalty for communication between different cores in the same cluster,} \\ w_{\text{intercluster}} &:= \text{penalty for communication between cores in different clusters.} \end{aligned}$$

For an explicitly defined communication path  $(c, d)$ , let  $q_{cd}^{\text{explicit}}$  be its specified penalty. The effective communication penalty  $q_{cd}$  between cores  $c, d \in C$  is:

$$q_{cd} = \begin{cases} q_{cd}^{\text{explicit}}, & \text{if an explicit communication path } (c, d) \text{ is defined,} \\ w_{\text{intra}}, & \text{if } c = d, \\ w_{\text{intercore}}, & \text{if } c \neq d \text{ and } \kappa(c) = \kappa(d), \\ w_{\text{intercluster}}, & \text{if } \kappa(c) \neq \kappa(d). \end{cases} \quad (3)$$

## 4 Decision Variables

$$\begin{array}{lll} y_{tc} \in \{0, 1\} & \forall t \in T, c \in C, & \text{equals 1 iff task } t \text{ is assigned to core } c, \\ x_{jc} \in \{0, 1\} & \forall j \in J, c \in C, & \text{equals 1 iff job } j \text{ is assigned to core } c, \\ s_j \geq 0 & \forall j \in J, & \text{start time of job } j, \\ f_j \geq 0 & \forall j \in J, & \text{finish time of job } j, \\ o_c^{\text{core}} \geq 0 & \forall c \in C, & \text{memory overflow on core } c, \\ o_k^{\text{cluster}} \geq 0 & \forall k \in K, & \text{memory overflow on cluster } k. \end{array}$$

For each unordered pair of distinct jobs  $\{j, j'\}$  with  $j < j'$ :

$$u_{jj'} \in \{0, 1\} \quad \forall j, j' \in J, j < j', \quad \text{ordering variable used for non-overlap constraints.}$$

For each task dependency  $(t_1, t_2) \in D_{\text{task}}$  and eligible core pair  $c \in E_{t_1}, d \in E_{t_2}$ :

$$z_{t_1 t_2 c d} \in \{0, 1\} \quad \forall (t_1, t_2) \in D_{\text{task}}, c \in E_{t_1}, d \in E_{t_2}, \quad \text{equals 1 iff } t_1 \text{ is assigned to } c \text{ and } t_2 \text{ is assigned to } d.$$

## 5 Objective Function

The objective minimizes weighted core-memory overflow, weighted cluster-memory overflow, and communication penalties along task dependency edges. Schedule makespan is managed strictly through release times and deadlines rather than objective optimization.

$$\min \quad \lambda_{\text{core}} \sum_{c \in C} o_c^{\text{core}} + \lambda_{\text{cluster}} \sum_{k \in K} o_k^{\text{cluster}} + \sum_{(t_1, t_2) \in D_{\text{task}}} \sum_{c \in E_{t_1}} \sum_{d \in E_{t_2}} q_{cd} z_{t_1 t_2 c d}. \quad (4)$$

## 6 Constraints

### 6.1 Task Assignment & Partitioned Scheduling

Each task must be assigned exactly once to one of its eligible cores:

$$\sum_{c \in E_t} y_{tc} = 1 \quad \forall t \in T. \quad (5)$$

Assignments to ineligible cores are forbidden:

$$y_{tc} = 0 \quad \forall t \in T, \forall c \in C \setminus E_t. \quad (6)$$

To enforce partitioned scheduling, every job must execute on the core to which its parent task is assigned:

$$x_{jc} = y_{\tau(j)c} \quad \forall j \in J, \forall c \in C. \quad (7)$$

### 6.2 Timing & Deadline Constraints

Each job may not start before its release time:

$$s_j \geq r_j \quad \forall j \in J. \quad (8)$$

If a chain root job has strict jitter constraints, its start time may optionally be bounded:

$$s_j \leq r_j + \text{jitter} \quad (\text{for applicable root jobs}). \quad (9)$$

The finish time of each job is determined by its start time and the processing time induced by the selected core:

$$f_j = s_j + \sum_{c \in C} p_j \alpha_c x_{jc} \quad \forall j \in J. \quad (10)$$

Terminal jobs in a chain with an defined absolute deadline must finish prior to that deadline:

$$f_j \leq d_j \quad \forall j \in J \text{ with } d_j \neq \emptyset. \quad (11)$$

### 6.3 Precedence Constraints

For every job execution dependency, the successor job may only start after the predecessor job has finished:

$$s_{j'} \geq f_j \quad \forall (j, j') \in D_{\text{job}}. \quad (12)$$

### 6.4 Memory-Budget Constraints

Memory footprints are evaluated at the *Task* level.

Core-level memory use may exceed the core memory budget only through the non-negative overflow variable:

$$\sum_{t \in T} m_t y_{tc} \leq B_c^{\text{core}} + o_c^{\text{core}} \quad \forall c \in C. \quad (13)$$

Cluster-level memory use may exceed the cluster memory budget only through the non-negative overflow variable:

$$\sum_{t \in T} \sum_{c \in C_k} m_t y_{tc} \leq B_k^{\text{cluster}} + o_k^{\text{cluster}} \quad \forall k \in K. \quad (14)$$

## 6.5 Non-Overlap Constraints on Cores

For every unordered pair of jobs  $j, j' \in J$  with  $j < j'$ , and for every core  $c \in C$ , the model enforces that if both jobs are assigned to the same core, one must finish before the other starts.

$$s_{j'} \geq f_j - M(3 - u_{jj'} - x_{jc} - x_{j'c}) \quad \forall j, j' \in J, j < j', \forall c \in C. \quad (15)$$

$$s_j \geq f_{j'} - M(2 + u_{jj'} - x_{jc} - x_{j'c}) \quad \forall j, j' \in J, j < j', \forall c \in C. \quad (16)$$

Here,  $u_{jj'} = 1$  selects the ordering  $j$  before  $j'$ , while  $u_{jj'} = 0$  selects the ordering  $j'$  before  $j$ .

## 6.6 Communication-Penalty Linearization

For each task dependency  $(t_1, t_2) \in D_{\text{task}}$  and each eligible core pair  $c \in E_{t_1}, d \in E_{t_2}$ , the auxiliary variable  $z_{t_1 t_2 c d}$  represents the logical conjunction  $z_{t_1 t_2 c d} = y_{t_1 c} y_{t_2 d}$ .

Because the objective minimizes the sum over  $z_{t_1 t_2 c d}$  and all coefficients  $q_{cd}$  are non-negative, the product can be sufficiently linearized with the single lower-bound constraint:

$$z_{t_1 t_2 c d} \geq y_{t_1 c} + y_{t_2 d} - 1 \quad \forall (t_1, t_2) \in D_{\text{task}}, c \in E_{t_1}, d \in E_{t_2}. \quad (17)$$

## 7 Variable Domains

$$\begin{array}{ll} y_{tc} \in \{0, 1\} & \forall t \in T, c \in C, \\ x_{jc} \in \{0, 1\} & \forall j \in J, c \in C, \\ u_{jj'} \in \{0, 1\} & \forall j, j' \in J, j < j', \\ z_{t_1 t_2 c d} \in \{0, 1\} & \forall (t_1, t_2) \in D_{\text{task}}, c \in E_{t_1}, d \in E_{t_2}, \\ s_j, f_j \in [0, M] & \forall j \in J, \\ o_c^{\text{core}} \geq 0 & \forall c \in C, \\ o_k^{\text{cluster}} \geq 0 & \forall k \in K. \end{array}$$

## 8 Remarks

- **Decoupling of Tasks and Jobs:** The current architecture decouples memory footprints (Tasks) from temporal executions (Jobs). While multiple instances (Jobs) of a Task may exist in the schedule, the core's memory budget is only impacted once per assigned Task.
- **Constraint Solvers:** In practice, tools like CP-SAT natively handle intervals and 'NoOverlap' constraints, removing the need for manual Big-M transformations ( $u_{jj'}$ ). This document represents the formal mathematical equivalent.
- **Memory-Pressure Optimization Weight:** Memory budgets remain modeled as soft constraints to preserve model feasibility. The scalar weights  $\lambda_{\text{core}}$  and  $\lambda_{\text{cluster}}$  define the preferred optimization focus on memory pressure: values closer to 0 reduce the importance of memory-budget violations in the objective, while larger values increase the optimizer's preference for assignments that respect core- and cluster-level memory limits.